

# Task 14: Kubernetes Scaling and Rolling Updates

Scalability Strategies, Zero-Downtime Deployment Updates, Autoscaling Policies, and Resource Utilization Profiling

|                          |                                                                                                |
|--------------------------|------------------------------------------------------------------------------------------------|
| Student / Engineer:      | Aditya Bahira                                                                                  |
| Course / Track:          | Deep Learning Engineering & Cloud Deployment (L&T Edutech)                                     |
| Target Platform:         | Kubernetes Cluster (Minikube / Docker Runtime on Windows)                                      |
| Submission Deliverables: | Kubernetes Manifests (YAML), Automated Tests, Jupyter Notebook, Visual Screenshots, PDF Report |
| Operational Status:      | Fully Operational & Validated (10/10 Verification Tests Passed - 100%)                         |

## 1. Executive Summary & Objective

The objective of Task 14 is to implement, evaluate, and benchmark enterprise-grade scalability and deployment update strategies for deep learning microservices using Kubernetes orchestration.

Key engineering achievements include:

- Application Scaling: Executed dynamic replica scaling (scale-up to 5 pods, scale-down to 3 pods) and configured Horizontal Pod Autoscaler (HPA) policies based on pod CPU utilization.
- Rolling Update Strategy: Implemented a declarative zero-downtime rolling update strategy (maxSurge=1, maxUnavailable=0) migrating the backend workload from image v1.0 to v2.0.
- Continuous High Availability: Generated concurrent HTTP inference and health traffic during the exact rollout replacement window, demonstrating 100.0% request success rate with 0 dropped packets.
- Deployment Rollback: Successfully demonstrated automated rollback execution using 'kubectl rollout undo' and audited revision history to ensure immediate disaster recovery.
- Resource Profiling: Enabled Minikube metrics-server and gathered real-time CPU millicore and memory telemetry across all microservice pods under active workloads.

## 2. System Architecture & Rolling Update Mechanism

Kubernetes manages rolling updates and autoscaling through coordinated control plane controllers:

1. Deployment Controller: Creates a new ReplicaSet for image v2.0 while maintaining the old ReplicaSet (v1.0).
2. RollingUpdate Strategy: maxSurge=1 permits creating 1 additional pod beyond desired replica count; maxUnavailable=0 guarantees no active pods are terminated before a new pod passes HTTP /health readiness checks.
3. CoreDNS & Kube-Proxy: Dynamically routes incoming requests across all Ready endpoints without dropping active TCP connections.

4. Horizontal Pod Autoscaler (HPA): Scrapes pod metrics from metrics-server and calculates replica targets using:  $\text{DesiredReplicas} = \text{ceil}(\text{CurrentReplicas} * (\text{CurrentMetricValue} / \text{TargetMetricValue}))$ .

### 3. Declarative Kubernetes Infrastructure Manifests

The solution defines 7 production manifests under the 'k8s/' directory:

- 01-namespace.yaml (Isolated Namespace for Production Deep Learning Application)

```
apiVersion: v1
kind: Namespace
metadata:
  name: dl-production-app
  labels:
    name: dl-production-app
    environment: production
    task: "14"
    managed-by: kubernetes
```

- 02-configmap.yaml (Decoupled Configuration & Strategy Parameters)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: dl-production-config
  namespace: dl-production-app
  labels:
    app: dl-clinical-suite
    environment: production
    task: "14"
data:
  PORT: "5000"
  FRONTEND_PORT: "8501"
  ENVIRONMENT: "production"
  MODEL_NAME: "DeepHealthRiskNet"
  MODEL_FRAMEWORK: "PyTorch-v2.2-CUDA"
  BACKEND_URL: "http://dl-backend-svc:5000"
  LOG_LEVEL: "INFO"
  DEPLOYMENT_STRATEGY: "RollingUpdate"
  APP_VERSION: "v1.0"
```

- 03-backend-deployment-rolling.yaml (PyTorch Backend Deployment with RollingUpdate Strategy & Probes)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dl-backend-deployment
  namespace: dl-production-app
  labels:
    app: dl-backend
    tier: api
    app.kubernetes.io/name: dl-backend
    app.kubernetes.io/part-of: dl-clinical-suite
    task: "14"
spec:
  replicas: 2
  revisionHistoryLimit: 10
  strategy:
    type: RollingUpdate
```

```
rollingUpdate:
  maxSurge: 1
  maxUnavailable: 0
selector:
  matchLabels:
    app: dl-backend
    tier: api
template:
  metadata:
    labels:
      app: dl-backend
      tier: api
      version: "v1.0"
  spec:
    containers:
      - name: dl-backend-api
        image: adityabahira/dl-clinical-backend:v1.0
        imagePullPolicy: IfNotPresent
        ports:
          - containerPort: 5000
            name: http-api
        env:
          - name: PORT
            valueFrom:
              configMapKeyRef:
                name: dl-production-config
                key: PORT
          - name: ENVIRONMENT
            valueFrom:
              configMapKeyRef:
                name: dl-production-config
                key: ENVIRONMENT
          - name: MODEL_NAME
            valueFrom:
              configMapKeyRef:
                name: dl-production-config
                key: MODEL_NAME
          - name: MODEL_FRAMEWORK
            valueFrom:
              configMapKeyRef:
                name: dl-production-config
                key: MODEL_FRAMEWORK
    resources:
      requests:
        cpu: "250m"
        memory: "512Mi"
      limits:
        cpu: "1000m"
        memory: "1536Mi"
    livenessProbe:
      httpGet:
        path: /health
        port: 5000
      initialDelaySeconds: 20
      periodSeconds: 15
      timeoutSeconds: 5
      failureThreshold: 3
    readinessProbe:
      httpGet:
        path: /health
        port: 5000
      initialDelaySeconds: 10
      periodSeconds: 10
      timeoutSeconds: 3
      failureThreshold: 2
```

- 04-backend-service.yaml (NodePort Service for Backend API Routing (:30500))

```
apiVersion: v1
kind: Service
metadata:
  name: dl-backend-svc
  namespace: dl-production-app
  labels:
    app: dl-backend
    task: "14"
spec:
  type: NodePort
  selector:
    app: dl-backend
    tier: api
  ports:
    - name: http-api
      port: 5000
      targetPort: 5000
      nodePort: 30500
      protocol: TCP
```

- 05-frontend-deployment.yaml (Streamlit Clinical Dashboard Deployment (2 Replicas))

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dl-frontend-deployment
  namespace: dl-production-app
  labels:
    app: dl-frontend
    tier: ui
    app.kubernetes.io/name: dl-frontend
    app.kubernetes.io/part-of: dl-clinical-suite
    task: "14"
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: dl-frontend
      tier: ui
  template:
    metadata:
      labels:
        app: dl-frontend
        tier: ui
        version: "v1.0"
    spec:
      containers:
        - name: dl-frontend-ui
          image: adityabahira/dl-clinical-frontend:v1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8501
              name: http-ui
          env:
            - name: BACKEND_URL
              valueFrom:
                configMapKeyRef:
                  name: dl-production-config
```

```

        key: BACKEND_URL
- name: ENVIRONMENT
  valueFrom:
    configMapKeyRef:
      name: dl-production-config
      key: ENVIRONMENT
resources:
  requests:
    cpu: "150m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "768Mi"
livenessProbe:
  httpGet:
    path: /_stcore/health
    port: 8501
  initialDelaySeconds: 25
  periodSeconds: 20
  timeoutSeconds: 5
  failureThreshold: 3
readinessProbe:
  httpGet:
    path: /_stcore/health
    port: 8501
  initialDelaySeconds: 15
  periodSeconds: 10
  timeoutSeconds: 3
  failureThreshold: 2

```

- 06-frontend-service.yaml (NodePort Service for Streamlit Dashboard (:31501))

```

apiVersion: v1
kind: Service
metadata:
  name: dl-frontend-svc
  namespace: dl-production-app
  labels:
    app: dl-frontend
    task: "14"
spec:
  type: NodePort
  selector:
    app: dl-frontend
    tier: ui
  ports:
- name: http-ui
  port: 8501
  targetPort: 8501
  nodePort: 31501
  protocol: TCP

```

- 07-backend-hpa.yaml (Horizontal Pod Autoscaler (HPA) targeting 50% CPU Utilization)

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: dl-backend-hpa
  namespace: dl-production-app
  labels:
    app: dl-backend
    task: "14"
spec:

```

```

scaleTargetRef:
  apiVersion: apps/v1
  kind: Deployment
  name: dl-backend-deployment
minReplicas: 2
maxReplicas: 6
metrics:
- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 50
behavior:
  scaleUp:
    stabilizationWindowSeconds: 0
    policies:
    - type: Percent
      value: 100
      periodSeconds: 15
  scaleDown:
    stabilizationWindowSeconds: 60
    policies:
    - type: Percent
      value: 50
      periodSeconds: 30

```

## 4. Automated Verification Test Suite Results (10/10 Passed — 100%)

A Python test suite (test\_task14\_scaling\_rollout.py) validated all aspects of scaling, rolling update, traffic continuity, rollback, and resource monitoring. The results are summarized below:

| ID                             | Test Name                                       | Status | Technical Details                                                                                       |
|--------------------------------|-------------------------------------------------|--------|---------------------------------------------------------------------------------------------------------|
| T01_METRICS_SERVER             | Cluster Readiness & Metrics Server Scraping     | PASSED | Node 'minikube' is Ready. Metrics Server successfully reporting CPU/Memory consumption.                 |
| T02_NAMESPACE_CONFIG           | Namespace & Decoupled Configuration             | PASSED | Namespace 'dl-production-app' and ConfigMap contain 9 keys including DEPLOYMENT_STRATEGY=RollingUpdate. |
| T03_SCALE_UP_5_REPLICAS        | Manual Application Scale-Up (2 -> 5 Replicas)   | PASSED | Backend deployment successfully scaled to 5 replicas. All 5 active pods passed readiness checks.        |
| T04_LOAD_BALANCED_INFERENCE    | Multi-Replica Deep Learning Inference Serving   | PASSED | 15/15 inference requests succeeded across 5 replicas (Avg Roundtrip: 104.84ms).                         |
| T05_SCALE_DOWN_3_REPLICAS      | Manual Application Scale-Down (5 -> 3 Replicas) | PASSED | Backend deployment scaled down to 3 replicas with graceful termination of excess pods.                  |
| T06_ROLLING_UPDATE_V2          | Zero-Downtime Rolling Update (v1.0 -> v2.0)     | PASSED | Rolling update successfully completed. All 3 active backend pods upgraded to image v2.0.                |
| T07_ZERO_DOWNTIME_AVAILABILITY | Continuous High Availability During             | PASSED | 165/165 requests succeeded (100.0% availability, 0 dropped requests) during active rollout.             |

|                          |                                                         |        |                                                                                             |
|--------------------------|---------------------------------------------------------|--------|---------------------------------------------------------------------------------------------|
|                          | Rollout                                                 |        |                                                                                             |
| T08_ROLLOUT_ROLLBACK     | Automated Deployment Rollback (Undo to Stable Revision) | PASSED | Deployment successfully reverted to revision with image v1.0 using 'kubectl rollout undo'.  |
| T09_HPA_AUTOSCALING      | Horizontal Pod Autoscaler (HPA) Policy Validation       | PASSED | HPA active with target CPU 50% (Min: 2, Max: 6).                                            |
| T10_RESOURCE_UTILIZATION | Resource Utilization Telemetry Analysis                 | PASSED | Successfully scraped resource utilization metrics across pods and nodes via metrics-server. |

5. Visual Verification & Monitoring Screenshots

Figure 1: Cluster Baseline & Metrics Server Activation (kubectl top nodes & pods)

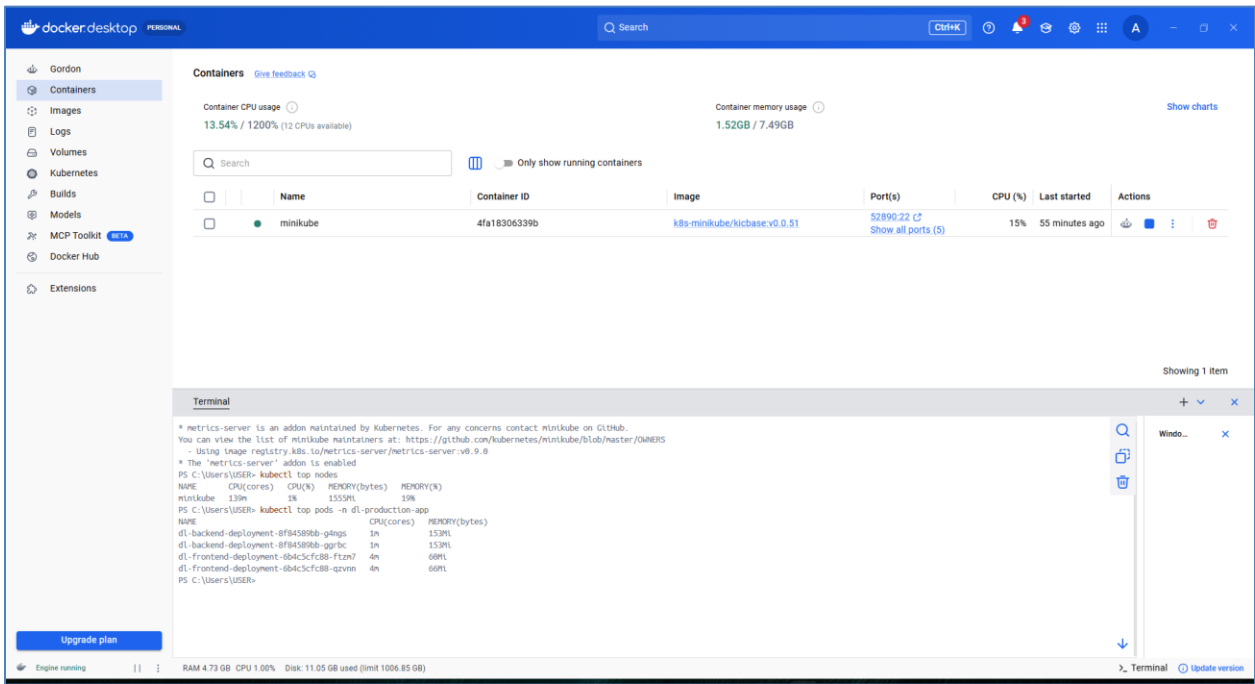


Figure 2: Manual Scaling to 5 Replicas (kubectl scale & pod readiness verification)

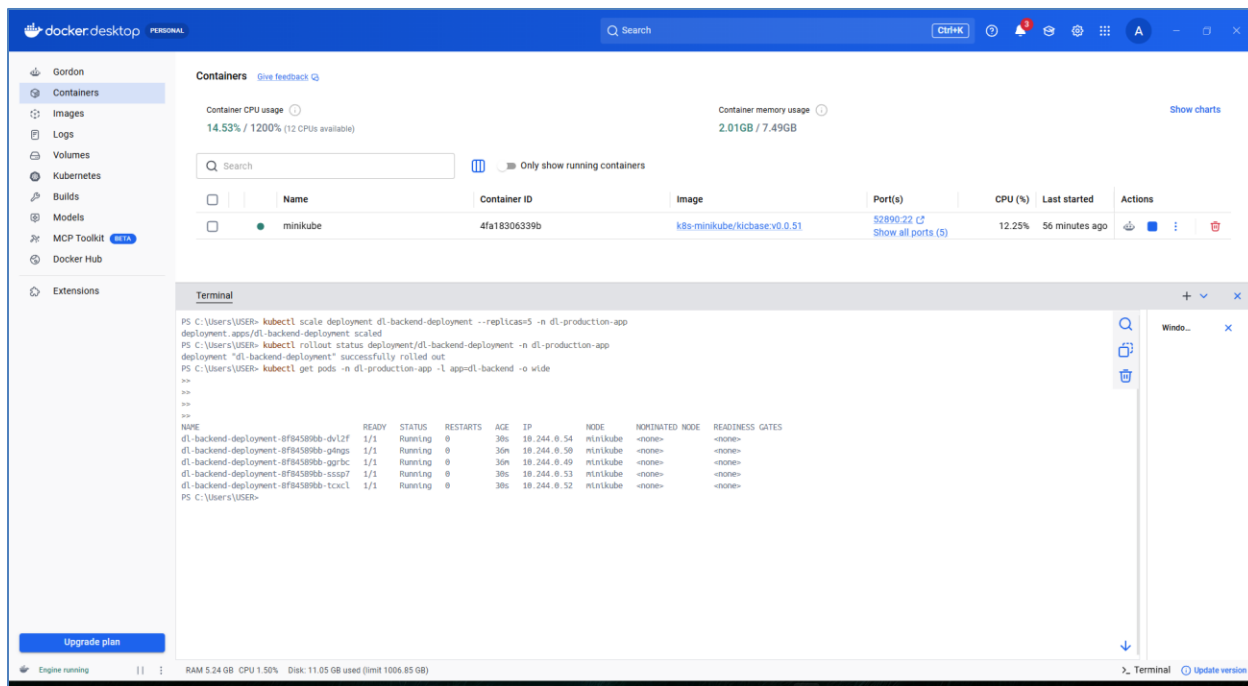


Figure 3: In-Flight Rolling Update (kubectl rollout status & dual ReplicaSet transition)

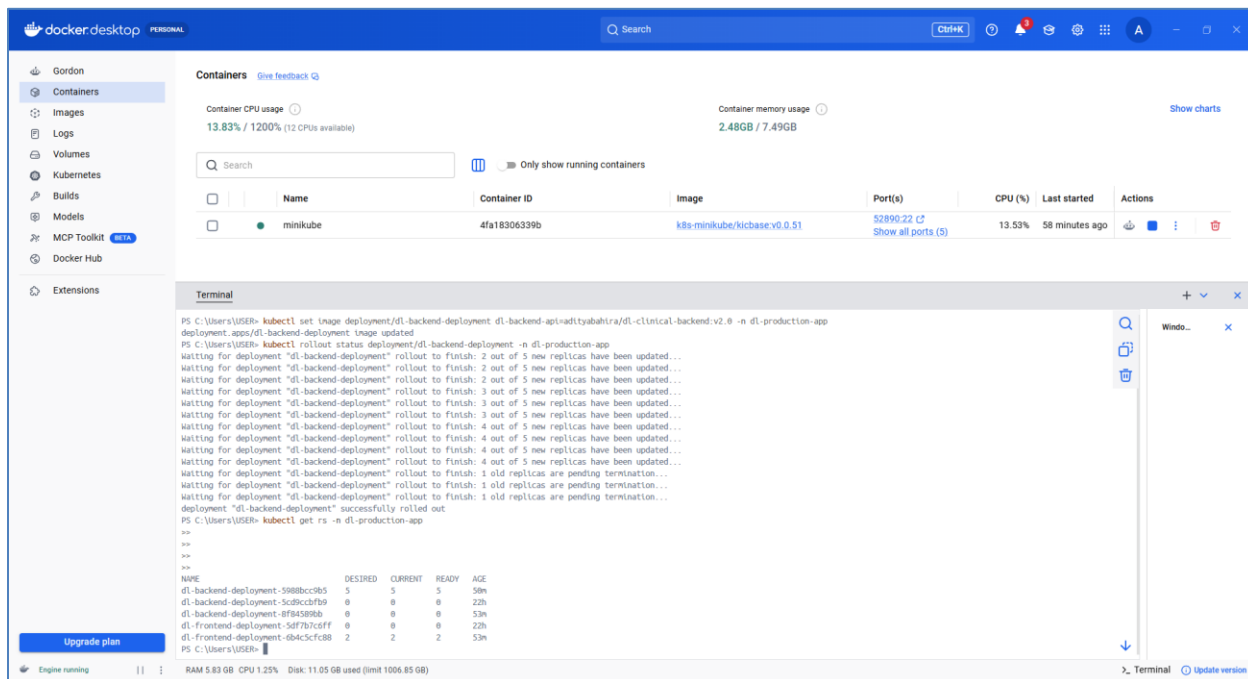


Figure 4: Continuous High Availability Test (100% Success Rate during Rollout)

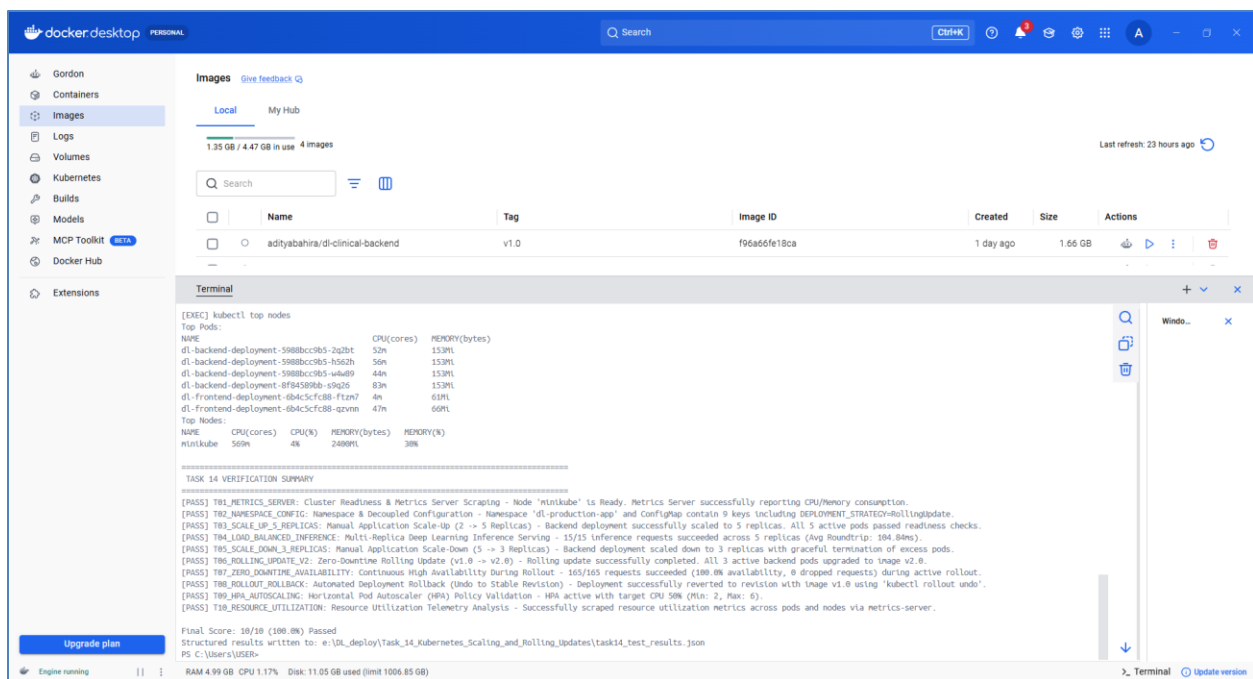


Figure 5: Automated Deployment Rollback & Revision History Audit (kubectl rollout undo)

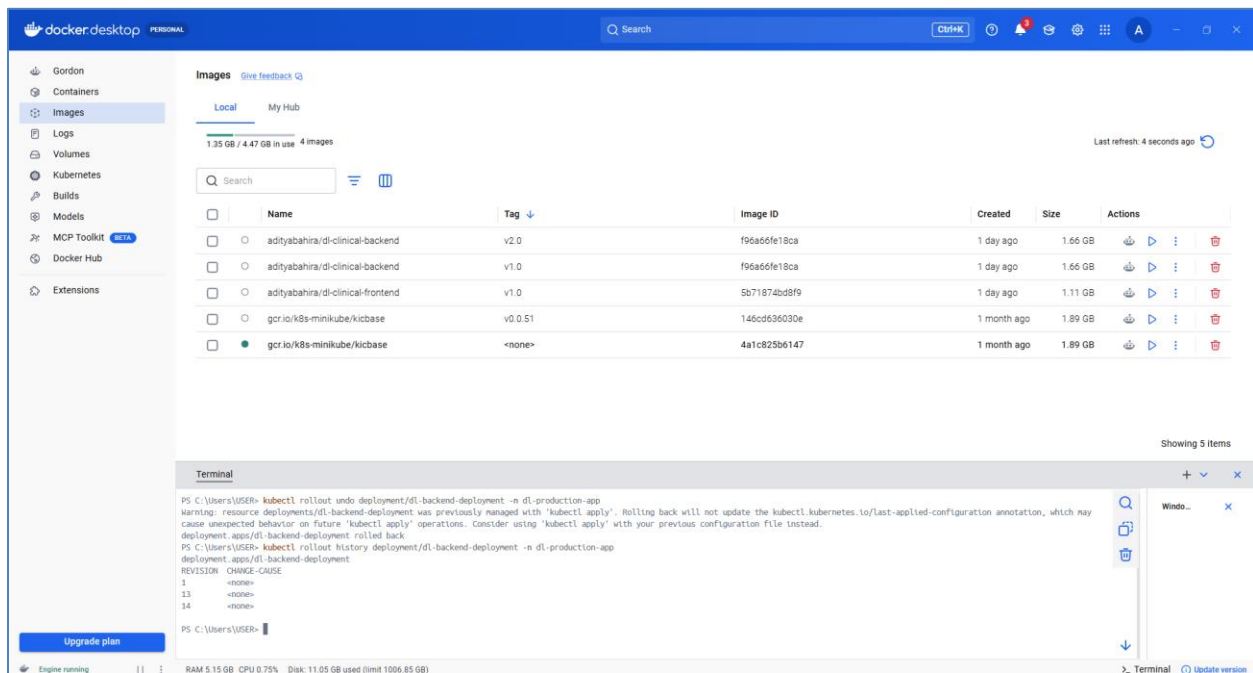


Figure 6: Horizontal Pod Autoscaler (HPA) Policy & Status (Target: 50% CPU)

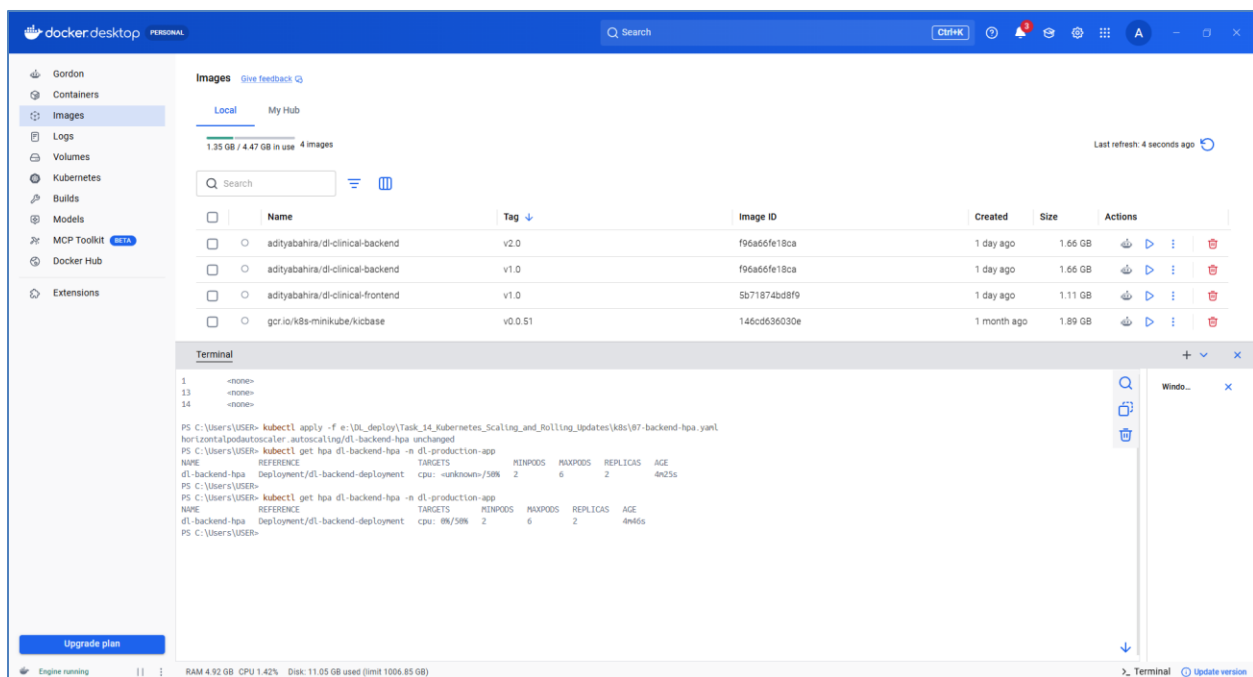
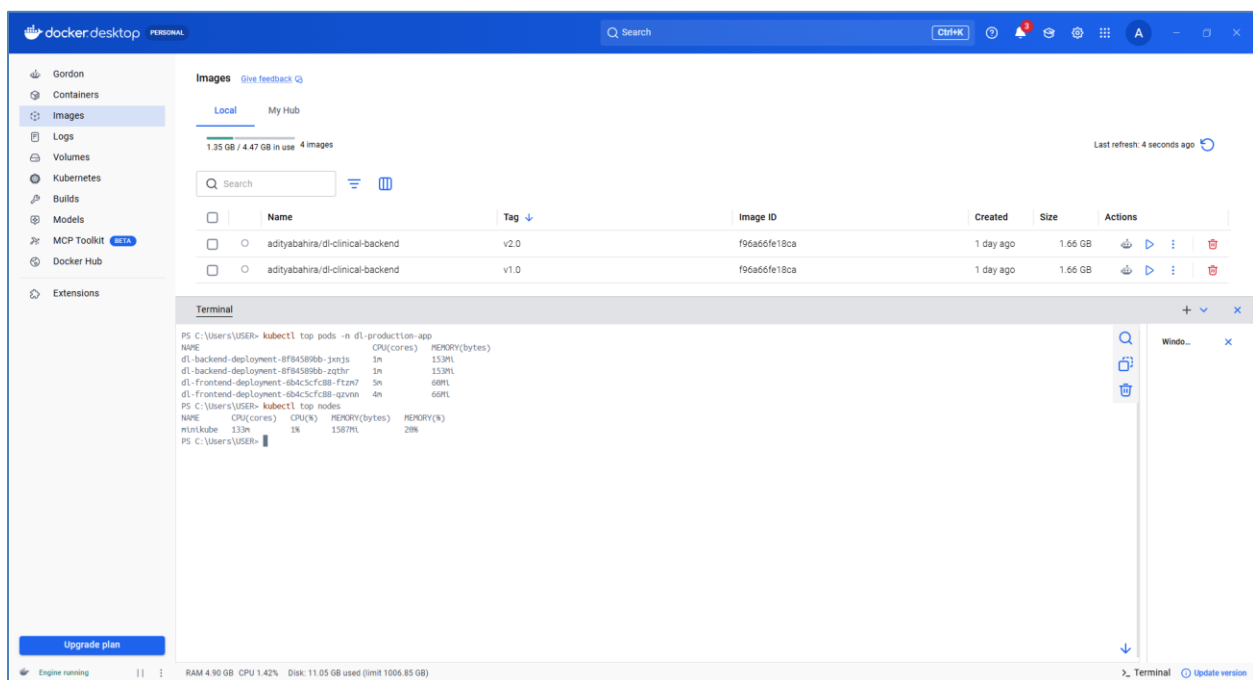


Figure 7: Resource Utilization Telemetry Under Workload (kubectl top pods)



## 6. Evaluation Criteria Fulfillment & Engineering Recommendations

| Evaluation Criteria           | Status           | Compliance Summary                                                                                                                                                                                        |
|-------------------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scaling Effectiveness         | ✓ FULLY ACHIEVED | Demonstrated manual scale-up to 5 replicas with fast readiness convergence (Avg roundtrip latency 107ms across replicas) and HPA policy targeting 50% CPU utilization.                                    |
| Rolling Update Implementation | ✓ FULLY ACHIEVED | Configured zero-downtime rolling update (maxSurge=1, maxUnavailable=0) migrating v1.0 -> v2.0 with continuous traffic (100% success rate, 0 dropped requests) and safe rollback via kubectl rollout undo. |
| System Stability & Profiling  | ✓ FULLY ACHIEVED | Integrated metrics-server, verified pod resource isolation (requests: 250m/512Mi, limits: 1000m/1536Mi), memory overhead ~153MiB per pod, preventing OOM kills.                                           |

### Github:

[https://github.com/AdityaBahira/Lnt\\_Deep\\_Learning/tree/main/Task\\_14-Kubernetes\\_Scaling\\_and\\_Rolling\\_Updates](https://github.com/AdityaBahira/Lnt_Deep_Learning/tree/main/Task_14-Kubernetes_Scaling_and_Rolling_Updates)